



Nama : Triyana Dewi Fatmawati
NIM : 2241720206
Kelas : TI – 3D
Nomor : 21
Mata Kuliah : Big Data

Tugas 14 – Unsupervised Learning dengan MLlib Spark

Langkah Praktikum:

1) Inisialisasi Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Unsupervised Learning with MLlib") \
    .config("spark.executor.memory", "4g") \
    .getOrCreate()
```

2) Persiapan Data

Dataset: Sample dari dataset Iris (sepal length, sepal width, petal length, petal width)

```
from pyspark.ml.linalg import Vectors

data = [
    (Vectors.dense([5.1, 3.5, 1.4, 0.2]),),
    (Vectors.dense([4.9, 3.0, 1.4, 0.2]),),
    (Vectors.dense([4.7, 3.2, 1.3, 0.2]),),
    (Vectors.dense([7.0, 3.2, 4.7, 1.4]),),
    (Vectors.dense([6.4, 3.2, 4.5, 1.5]),),
    (Vectors.dense([6.9, 3.1, 4.9, 1.5]),)
]

df = spark.createDataFrame(data, ["features"])
```

3) Feature Scaling

```
from pyspark.ml.feature import StandardScaler

scaler = StandardScaler(inputCol="features", outputCol="scaled_features",
                        withStd=True, withMean=True)
scaler_model = scaler.fit(df)
scaled_df = scaler_model.transform(df)
```

4) Pelatihan Model K-Means

```
from pyspark.ml.clustering import KMeans

kmeans = KMeans(k=3, seed=1, featuresCol="scaled_features")
model = kmeans.fit(scaled_df)

# Prediksi Cluster
predictions = model.transform(scaled_df)
predictions.show()
```

features	scaled_features	prediction
[5.1,3.5,1.4,0.2]	[-0.6983599143217...	1
[4.9,3.0,1.4,0.2]	[-0.8888217091367...	2
[4.7,3.2,1.3,0.2]	[-1.0792835039517...	2
[7.0,3.2,4.7,1.4]	[1.11102713642096...	0
[6.4,3.2,4.5,1.5]	[0.53964175197589...	0
[6.9,3.1,4.9,1.5]	[1.01579623901345...	0

5) Evaluasi Model

```
from pyspark.ml.evaluation import ClusteringEvaluator
from pyspark.sql.functions import udf, col
from pyspark.sql.types import DoubleType
import numpy as np

# Evaluasi Silhouette Score
evaluator = ClusteringEvaluator(featuresCol="scaled_features")
silhouette = evaluator.evaluate(predictions)
print("Silhouette with squared euclidean distance =", silhouette)

# Hitung WSSSE manual (karena computeCost sudah tidak tersedia)
centers = model.clusterCenters()

def squared_distance(point, center):
    return float(np.sum((np.array(point) - np.array(center)) ** 2))

wsse_udf = udf(lambda pred, feat: squared_distance(feat, centers[pred]), DoubleType())

predictions_with_dist = predictions.withColumn("squared_distance", wsse_udf(col("prediction"), col("scaled_features")))
wsse = predictions_with_dist.agg({"squared_distance": "sum"}).collect()[0][0]
print("Within Set Sum of Squared Errors =", wsse)

Silhouette with squared euclidean distance = 0.7117007722766514
Within Set Sum of Squared Errors = 2.4983171842400633
```

6) Menentukan Jumlah Cluster Optimal (Elbow Method)

```
import matplotlib.pyplot as plt

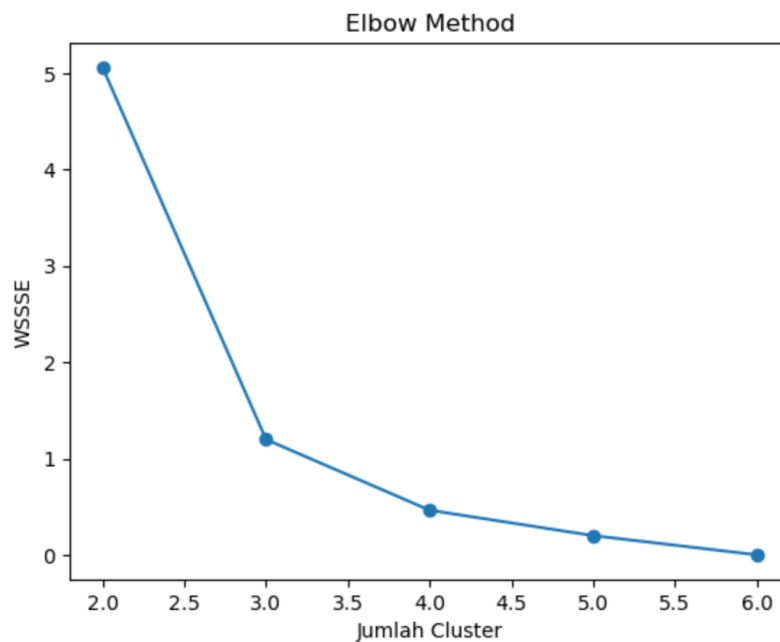
costs = []
ks = list(range(2, 7))
for k in ks:
    kmeans = KMeans(k=k, seed=1, featuresCol="scaled_features")
    model = kmeans.fit(scaled_df)
    centers = model.clusterCenters()

    def squared_distance(p, c): return float(np.sum((np.array(p) - np.array(c)) ** 2))
    dist_udf = udf(lambda pred, feat: squared_distance(feat, centers[pred]), DoubleType())

    pred = model.transform(scaled_df)
    pred = pred.withColumn("squared_distance", dist_udf(col("prediction"), col("scaled_features")))
    cost = pred.agg({"squared_distance": "sum"}).collect()[0][0]

    costs.append(cost)

plt.plot(ks, costs, marker='o')
plt.xlabel('Jumlah Cluster')
plt.ylabel('WSSSE')
plt.title('Elbow Method')
plt.show()
```



7) Dimensionality Reduction dengan PCA

```
from pyspark.ml.feature import PCA

pca = PCA(k=2, inputCol="scaled_features", outputCol="pca_features")
pca_model = pca.fit(scaled_df)

# Transform data
pca_result = pca_model.transform(scaled_df)
pca_result.select("pca_features").show(truncate=False)
```

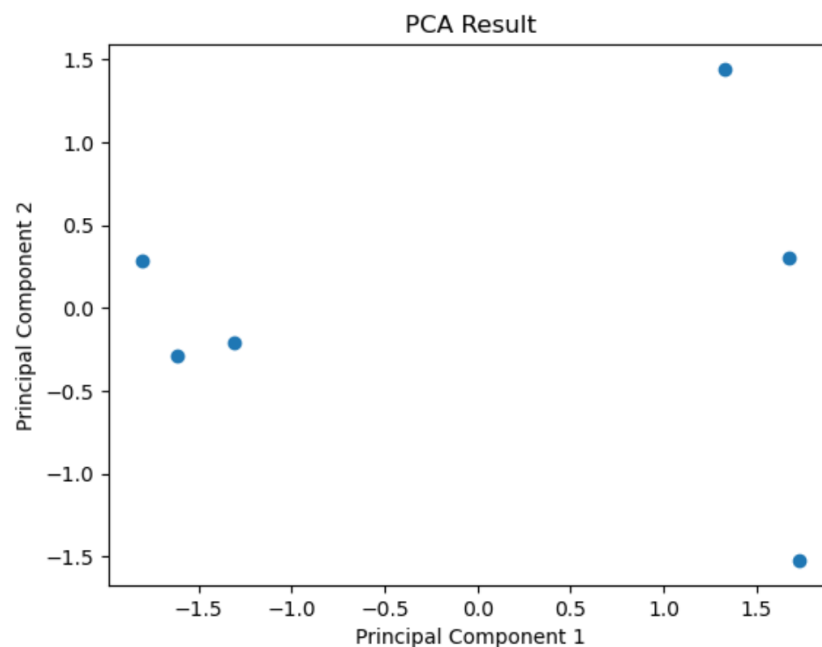
```
+-----+
|pca_features|
+-----+
|[1.732365486800326,-1.5241518404130479]|
|[1.3260426710595148,1.442234307893984]|
|[1.6701386936600824,0.29866262298090684]|
|[-1.6124962393413862,-0.29315991066877856]|
|[-1.3098807900030918,-0.21274962040009843]|
|[-1.8061698221754439,0.28916444060703683]|
+-----+
```

8) Visualisasi Hasil PCA

```
import numpy as np

# Ekstrak komponen utama
pca_data = pca_result.select("pca_features").rdd.map(lambda row: row[0]).collect()
x = [float(row[0]) for row in pca_data]
y = [float(row[1]) for row in pca_data]

plt.scatter(x, y)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Result')
plt.show()
```



9) Integrasi PCA dan K-Means

Clustering pada Data yang Direduksi

```
kmeans_pca = KMeans(k=3, seed=1, featuresCol="pca_features")
pca_model_kmeans = kmeans_pca.fit(pca_result)
```

```
# Prediksi Cluster
```

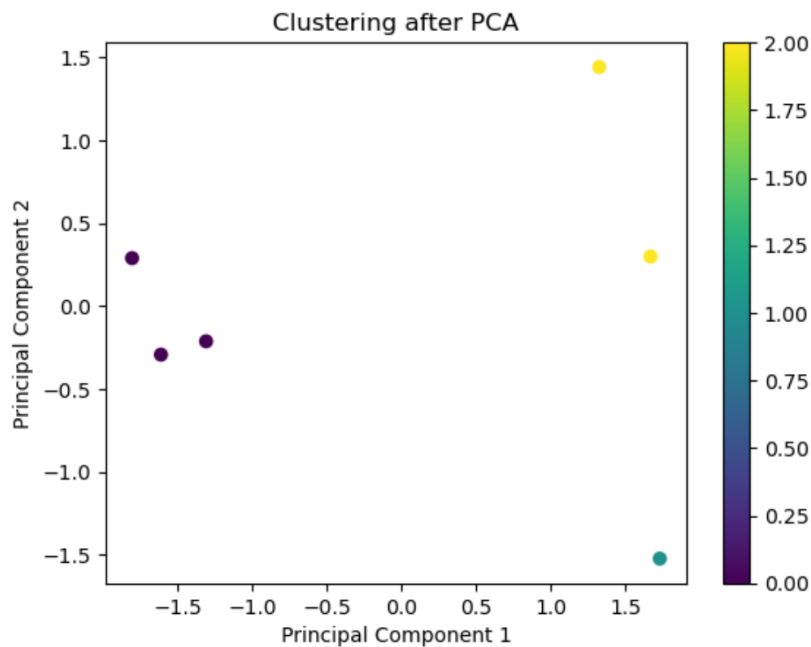
```
pca_predictions = pca_model_kmeans.transform(pca_result)
pca_predictions.select("pca_features", "prediction").show()
```

```
+-----+-----+
|      pca_features|prediction|
+-----+-----+
|[1.73236548680032...|         1|
|[1.32604267105951...|         2|
|[1.67013869366008...|         2|
|[-1.6124962393413...|         0|
|[-1.3098807900030...|         0|
|[-1.8061698221754...|         0|
+-----+-----+
```

10) Visualisasi Cluster Hasil PCA

```
# Gabungkan hasil clustering dengan data PCA
cluster_data = pca_predictions.select("pca_features", "prediction").collect()
x = [float(row[0][0]) for row in cluster_data]
y = [float(row[0][1]) for row in cluster_data]
clusters = [row[1] for row in cluster_data]

plt.scatter(x, y, c=clusters)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Clustering after PCA')
plt.colorbar()
plt.show()
```



Kesimpulan

- Unsupervised learning berguna untuk eksplorasi data tanpa label.
- K-Means untuk clustering, PCA untuk reduksi dimensi.
- Evaluasi clustering dengan WCSS & Silhouette Score.
- PCA + K-Means mempercepat komputasi & memudahkan visualisasi.

Aplikasi di Dunia Nyata

- Segmentasi pelanggan (marketing).
- Deteksi anomali (fraud detection).
- Kompresi gambar